FORESIGHT — Demo Video Narration (read aloud, ~4 min)

Record your screen (Win+G / Loom / OBS), read this over it, then upload unlisted to YouTube or Drive.

[0:00 — Intro · show the live dashboard]

Hi, I'm walking you through Project FORESIGHT — a demand and inventory intelligence system I built for NorthBay Living, a direct-to-consumer home and lifestyle brand. The problem: every month they stock out of their best-sellers and sit on slow movers, because they plan inventory on gut feel. FORESIGHT turns their sales history into a weekly demand forecast and an early-warning system that tells the team exactly what to reorder, what to clear, and what to leave alone.

[0:30 — GitHub repo · show github.com/SR005/foresight]

Everything lives in this GitHub repo. The `src` folder has the reproducible pipeline — data cleaning, the forecast model, and the risk scoring. The whole thing re-runs from raw data with a single command, `python src/run_all.py`. The README documents the problem, the data, and the honest accuracy results.

[1:00 — Data & pipeline · show README or eda_memo.md]

The data is about a million real transaction rows over two years. My pipeline cleans it — removing cancellations, returns, duplicates, and non-product codes — keeping 94% of rows, and every cleaning decision is logged with a reason. From that it builds a clean star schema and a weekly sales panel for the top 200 SKUs, which is NorthBay's active assortment.

[1:40 — The forecast · show the forecast chart / backtest numbers]

For forecasting, I followed the professional workflow: first build a seasonal-naive

baseline, then only trust a complex model if it beats that baseline honestly. My LightGBM model scores a WAPE of 0.56 versus the baseline's 0.89 — that's about 37% more accurate. And critically, it's validated with rolling-origin backtesting with no data leakage, so that accuracy is real, not inflated. Each forecast comes with an 80% confidence interval.

[2:20 — Risk scoring & decisioning grid · show the dashboard]

A forecast alone doesn't tell the team what to do — that's what the risk layer does. It combines each forecast with the inventory position to score stockout and overstock risk, and places every SKU on this decisioning grid. Top-left, in red, is 'reorder now.' Bottom-right, in indigo, is 'markdown and clear.' Green is healthy. Every SKU gets a recommended action and the rupees at stake, so the team can prioritise by cash impact.

[2:50 — Dashboard interaction · click a filter and a SKU]

The dashboard is built for the operations team to use without a data scientist. I can filter by category or risk quadrant here on the left. This prioritised action list shows exactly what to deal with first, sorted by rupees at stake. And if I pick any SKU, I see its history, the forecast, the baseline, and its recommended action right here.

[3:20 — The numbers that matter · show the KPI cards]

The bottom line for NorthBay: about 372,000 rupees of capital is locked in overstocked SKUs they should clear, and about 18,000 rupees of sales are at risk from items about to stock out. That's the business case — real money, tied to specific actions the team can take this week.

[3:45 — Executive readout & close · show the deck]

Finally, I packaged this into an executive readout for the Head of Operations and Finance — leading with the rupee impact, the recommended actions, and an honest section on accuracy and limitations. To sum up: FORESIGHT is a reproducible forecast that beats its baseline, a transparent risk layer that drives clear decisions, and a live dashboard the team can actually use. Thanks for

watching.

What to have open (tabs) before you hit record: 1. GitHub repo — github.com/SR005/foresight 2. Live dashboard — the Streamlit URL 3. Executive deck — reports/executive_readout.pptx (or the PDF)

Speak slowly; this runs ~3.5–4 min at a natural pace. Short on time? Drop the [3:20] KPI section.